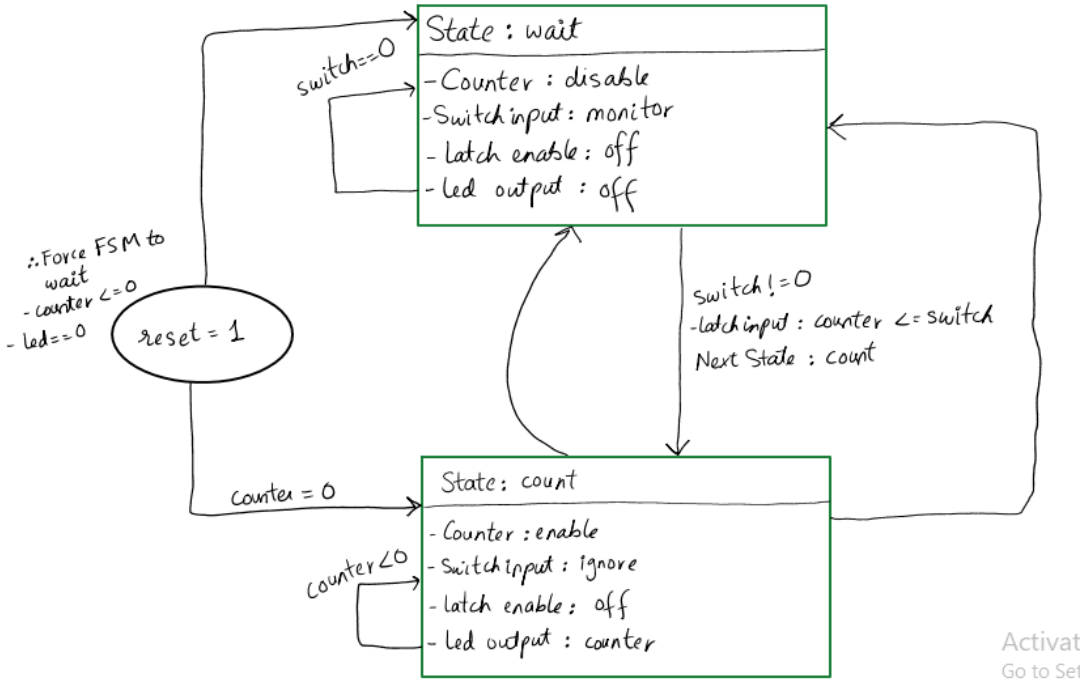
Lab 05 – Worksheet

Name: Areeba Izhar | Shaheer Qureshi	ID: ai09625 | sq09647	Section: T1

Note: Assumptions and logics should be explained separately in tasks after the task results.

Task 1

FSM:



Comments

Our FSM is a Moore Finite State Machine for a decrement counter. In the Input-Waiting State, the FSM monitors FPGA switches and remains idle if the input is zero. Upon detecting a non-zero value, the FSM captures the input and transitions to the Countdown State. Here the counter decrements toward zero while the LEDs reflect the current value and new switch inputs are ignored. The FSM automatically resets to the waiting state once the counter reaches zero or if the hardware reset button is pressed.

Task 2

Modules + Testbench:

Clock Divider:

```
`timescale 1ns / 1ps
module clock_divider (
    input wire clk_in,    // The super fast 100MHz board clock
    input wire rst,       // The reset button
    output reg clk_out    // Our new slow 1Hz clock
);

    // A register big enough to hold the number 50,000,000
    reg [25:0] count = 0;

    // We toggle the output every 50 Million ticks.
    // 50M High + 50M Low = 100M total ticks = 1 full second.

    // TEMPORARILY CHANGED FOR SIMULATION ONLY!
    // Change this back to 50_000_000 - 1 before programming the board!
    localparam MAX_COUNT = 2;
    //localparam MAX_COUNT = 50_000_000 - 1;

    always @(posedge clk_in) begin
        if (rst) begin
            count <= 0;
            clk_out <= 0;
        end
        else if (count == MAX_COUNT) begin
            count <= 0;
            clk_out <= ~clk_out; // Flip the slow clock opposite of what it was
        end
        else begin
            count <= count + 1;
        end
    end

endmodule
```

Top fsm:

```
`timescale 1ns / 1ps
module top_fsm_system (
    input wire clk,
    input wire pbin,
    input wire [15:0] physical_sw,
    output wire [15:0] physical_leds
```

```
);

    // DEBOUNCER (Cleans up the physical reset button signal)
    wire rst_clean;
    debouncer rst_db (.clk(clk),
        .pbin(pbin), .pbout(rst_clean)); //generated a clean signal

    wire [31:0] switch_data; // hold the value read from the switches

    leds switch_reader (.clk(clk), .rst(rst_clean),
        .btns(16'd0),// Not used for this FSM
        .writeData(32'd0),// We don't write to switches
        .writeEnable(1'b0),// Disabled
        .readEnable(1'b1),// Always ON so we can monitor switches
        .memAddress(30'd0),
        .switches(physical_sw),// Plug in the physical switches
        .readData(switch_data)// output data
        );

    reg [31:0] led_write_data = 32'd0; // counter value here

    switches led_writer (.clk(clk), .rst(rst_clean),
        .writeData(led_write_data),
        .writeEnable(1'b1),         // Always ON so LEDs update instantly
        .readEnable(1'b0), .memAddress(30'd0),
        .readData(),                // Ignored
        .leds(physical_leds)
    );
    wire slow_clk;
    clock_divider ticker (
        .clk_in(clk),          // Feed it the 100MHz fast clock
        .rst(rst_clean),       // Feed it the clean reset signal
        .clk_out(slow_clk)     // It spits out the 1Hz slow clock!
    );
    // FSM AND COUNTER LOGIC
    localparam WAIT  = 1'b0;
    localparam COUNT = 1'b1;
    reg state = WAIT;
    reg [15:0] counter = 0;

    // Synchronus
    always @(posedge slow_clk or posedge rst_clean) begin
        // SYNCHRONOUS RESET
        if (rst_clean == 1'b1) begin
            state <= WAIT;
            counter <= 16'd0;
            led_write_data <= 32'd0; // Turn off LEDs
        end
```

```
        else begin
            case (state)
                // STATE 0: WAIT (Idling and Monitoring)
                WAIT: begin
                    // If switches are NOT zero, latch them and start
                    if (switch_data[15:0] != 16'd0) begin
                        counter <= switch_data[15:0];// Latch to counter
                        led_write_data <= switch_data;// Send to LEDs
                        state <= COUNT;// Transition State
                    end
                    else begin
                        counter <= 16'd0;
                        led_write_data <= 32'd0;
                    end
                end
                // STATE 1: COUNT (Decrementing until 0)
                COUNT: begin
                    if (counter > 16'd0) begin
                        counter <= counter - 16'd1;// Decrement
                        led_write_data <= (counter - 16'd1);// Update LEDs
                    end
                    // hit zero so, go back to WAIT
                    if (counter == 16'd1 || counter == 16'd0) begin
                        state <= WAIT;
                    end
                end
            endcase
        end
    end
endmodule
```

Leds:

```
`timescale 1ns/1ps
module leds(
    input clk,
    input rst,
    input [15:0] btns,
    input [31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    input [15:0] switches,
    output [31:0] readData
    );
    assign readData = {16'd0, switches};
endmodule
```

Debouncer:

```
`timescale 1ns/1ps
module debouncer(
    input clk,
    input pbin,
    output pbout
    );
    assign pbout = pbin;
endmodule
```

Switches:

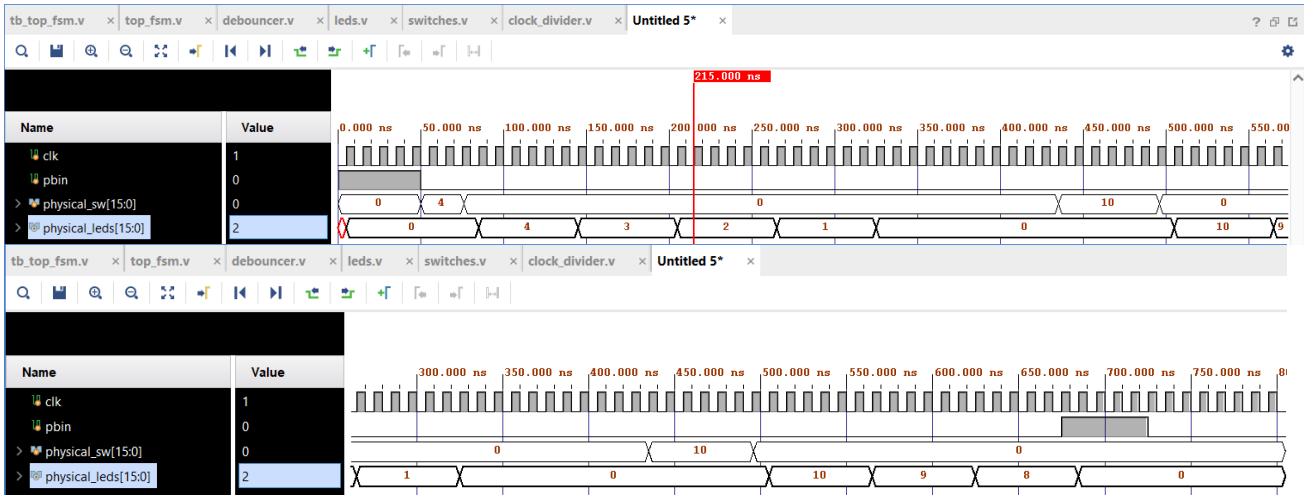
```
`timescale 1ns/1ps
module switches(
    input clk,
    input rst,
    input [31:0] writeData,
    input writeEnable,
    input readEnable,
    input [29:0] memAddress,
    output reg [31:0] readData = 0,
    output reg [15:0] leds
    );
    // Emergency LED writing logic
    always @(posedge clk) begin
        if (rst)
            leds <= 16'd0;
        else if (writeEnable)
            leds <= writeData[15:0];
    end
endmodule
```

Testbench:

```
`timescale 1ns / 1ps
module tb_fsm_system();
    reg clk;
    reg pbin;
    reg [15:0] physical_sw;
    wire [15:0] physical_leds;

    top_fsm_system uut (
        .clk(clk),
        .pbin(pbin),
        .physical_sw(physical_sw),
        .physical_leds(physical_leds)
    );

    // 100MHz Fast Clock Heartbeat
    always begin
```

```
    #5 clk = ~clk;
  end

  initial begin
    // --- STARTUP ---
    clk = 0;
    pbin = 1;       // Hold down Reset
    physical_sw = 16'd0;

    // Wait a few fast clock cycles, then release reset
    #50;
    pbin = 0;

    // --- TEST 1: The Natural Countdown ---
    physical_sw = 16'd4;

    // Wait until the FSM's slow clock ticks so it can grab the '4'
    @(posedge uut.slow_clk);
    #1; // Tiny delay to let the wires settle

    physical_sw = 16'd0; // Turn switches off

    // Tell the testbench to wait for 6 slow clock ticks
    // It will count: 4 -> 3 -> 2 -> 1 -> WAIT
    repeat(6) @(posedge uut.slow_clk);

    // --- TEST 2: The Emergency Stop ---
    physical_sw = 16'd10;
    @(posedge uut.slow_clk);
    #1;
    physical_sw = 16'd0;

    // Let it count down exactly 3 times (10 -> 9 -> 8)
    repeat(3) @(posedge uut.slow_clk);

    // SMASH THE RESET BUTTON!
    pbin = 1;
    #50; // Hold it for a bit
    pbin = 0;

    // Wait a couple more slow ticks to prove the LEDs stay OFF
    repeat(2) @(posedge uut.slow_clk);

    // End simulation
    $finish;
  end
endmodule
```

Results:



Task 3

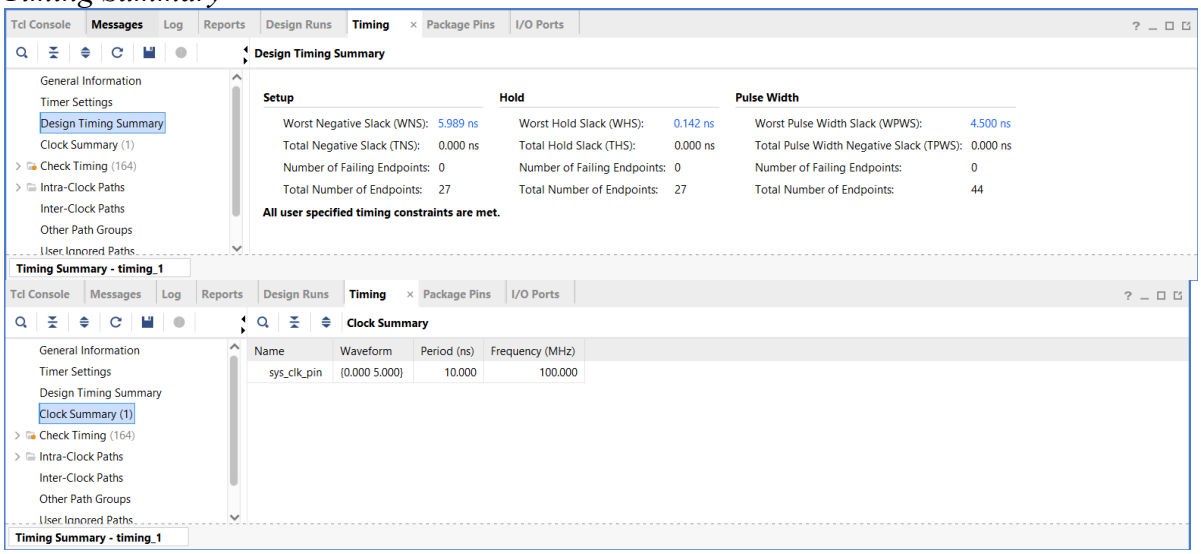
XDC file

```
## Clock signal
set_property PACKAGE_PIN W5 [get_ports clk]
set_property IOSTANDARD LVCMOS33 [get_ports clk]
create_clock -period 10.000 -name sys_clk_pin -waveform {0.000 5.000} -add
[get_ports clk]

## Switches
set_property PACKAGE_PIN V17 [get_ports {physical_sw[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[0]}]
set_property PACKAGE_PIN V16 [get_ports {physical_sw[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[1]}]
set_property PACKAGE_PIN W16 [get_ports {physical_sw[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[2]}]
set_property PACKAGE_PIN W17 [get_ports {physical_sw[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[3]}]
set_property PACKAGE_PIN W15 [get_ports {physical_sw[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[4]}]
set_property PACKAGE_PIN V15 [get_ports {physical_sw[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[5]}]
set_property PACKAGE_PIN W14 [get_ports {physical_sw[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[6]}]
set_property PACKAGE_PIN W13 [get_ports {physical_sw[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[7]}]
set_property PACKAGE_PIN V2 [get_ports {physical_sw[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[8]}]
set_property PACKAGE_PIN T3 [get_ports {physical_sw[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[9]}]
set_property PACKAGE_PIN T2 [get_ports {physical_sw[10]}]
```

```
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[10]}]
set_property PACKAGE_PIN R3 [get_ports {physical_sw[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[11]}]
set_property PACKAGE_PIN W2 [get_ports {physical_sw[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[12]}]
set_property PACKAGE_PIN U1 [get_ports {physical_sw[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[13]}]
set_property PACKAGE_PIN T1 [get_ports {physical_sw[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[14]}]
set_property PACKAGE_PIN R2 [get_ports {physical_sw[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_sw[15]}]


## LEDs
set_property PACKAGE_PIN U16 [get_ports {physical_leds[0]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[0]}]
set_property PACKAGE_PIN E19 [get_ports {physical_leds[1]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[1]}]
set_property PACKAGE_PIN U19 [get_ports {physical_leds[2]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[2]}]
set_property PACKAGE_PIN V19 [get_ports {physical_leds[3]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[3]}]
set_property PACKAGE_PIN W18 [get_ports {physical_leds[4]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[4]}]
set_property PACKAGE_PIN U15 [get_ports {physical_leds[5]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[5]}]
set_property PACKAGE_PIN U14 [get_ports {physical_leds[6]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[6]}]
set_property PACKAGE_PIN V14 [get_ports {physical_leds[7]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[7]}]
set_property PACKAGE_PIN V13 [get_ports {physical_leds[8]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[8]}]
set_property PACKAGE_PIN V3 [get_ports {physical_leds[9]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[9]}]
set_property PACKAGE_PIN W3 [get_ports {physical_leds[10]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[10]}]
set_property PACKAGE_PIN U3 [get_ports {physical_leds[11]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[11]}]
set_property PACKAGE_PIN P3 [get_ports {physical_leds[12]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[12]}]
set_property PACKAGE_PIN N3 [get_ports {physical_leds[13]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[13]}]
set_property PACKAGE_PIN P1 [get_ports {physical_leds[14]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[14]}]
set_property PACKAGE_PIN L1 [get_ports {physical_leds[15]}]
set_property IOSTANDARD LVCMOS33 [get_ports {physical_leds[15]}]
```

```
## Buttons
## Center Button (btnC) mapped to pbin
set_property PACKAGE_PIN U18 [get_ports pbin]
set_property IOSTANDARD LVCMOS33 [get_ports pbin]
```

Timing Summary



Interpretation:

Slack (MET) : 5.989ns
Required Time : 10.000ns
Path Delay : 4.011ns

It interprets that the value is +ve thus it's a PASS (good design)

Task 4

RTL Schematic:

(attached after this page)

Results

Link

Lab 05 – Designing FSM Using FPGA Switches and LEDs
Assessment Rubrics

Points Distribution

Task No.	LR2 Code	LR 5 Results	LR11 Design	AR7
Prelab		/15		/10 & /10
Task 1	-	-	/10	
Task 2	/15	/10		
Task 3		/15		
Task 4		/15		
Total Points	/80			/20
CLO Mapped	CLO 1			

For description of different levels of the mapped rubrics, please refer the provided Lab Evaluation Assessment Rubrics and Affective Domain Assessment Rubrics.

#	Assessment Elements	Level 1: Unsatisfactory Points 0-1	Level 2: Developing Points 2	Level 3:Good Points 3	Level 4:Exemplary Points 4
LR2	Program/Code / Simulation Model/ Network Model	Program/code/simulation model/network model does not implement the required functionality and has several errors. The student is not able to utilize even the basic tools of the software.	Program/code/simulation model/network model has some errors and does not produce completely accurate results. Student has limited command on the basic tools of the software.	Program/code/simulation model/network model gives correct output but not efficiently implemented or implemented by computationally complex routine.	Program/code/simulation /network model is efficiently implemented and gives correct output. Student has full command on the basic tools of the software.
LR5	Results & Plots	Figures/ graphs / tables are not developed or are poorly constructed with erroneous results. Titles, captions, units are not mentioned. Data is presented in an obscure manner.	Figures, graphs and tables are drawn but contain errors. Titles, captions, units are not accurate. Data presentation is not too clear.	All figures, graphs, tables are correctly drawn but contain minor errors or some of the details are missing.	Figures / graphs / tables are correctly drawn and appropriate titles/captions and proper units are mentioned. Data presentation is systematic.
AR9	Report	All the in-lab tasks are not included in report and / or the report is submitted too late.	Most of the tasks are included in report but are not well explained. All the necessary figures	Good summary of most the in-lab tasks is included in report. The work is supported by figures and	Detailed summary of the in-lab tasks is provided. All tasks are included and explained well. Data is presented clearly

			/ plots are not included. Report is submitted after due date.	plots with explanations. The report is submitted timely.	including all the necessary figures, plots and tables.